EE5203 L06 Practice Worksheet

EXTI, NVIC, callbacks, and debounce

Basri Erdogan

Expected time: 30-45 minutes **Policy:** Attempt every question before consulting the worked solutions. The graded homework (posted in CATS) contains the repair-and-extend work; this worksheet is your rehearsal for it.

Part A - The path and its names

1. Put these stations in the order a B1 press visits them, and give each one's one-line job:

   `NVIC` · `HAL_GPIO_EXTI_Callback` · falling edge on PC13 · `EXTI15_10_IRQHandler` · vector table · EXTI13 edge detector · `HAL_GPIO_EXTI_IRQHandler` · `EXTI15_10_IRQn`

2. Why do PC13, PB13, and PA13 all map to EXTI13 — and why can only one of them use it at a time? Name the register field that stores the choice and its value in our configuration.

3. Lines 10-15 share one IRQ. If a design used interrupts on both PC13 and PB11, describe what the handler would have to do that our single-button handler does not.

Part B - Configuration and registers

4. For each of the three CubeMX choices on the lab guide, name the register bit(s) the generated code writes as a result:

   a. PC13 = `GPIO_EXTI13`, falling mode;
   b. no pull-up / no pull-down;
   c. NVIC checkbox for `EXTI line[15:10]`.

5. `EXTI->PR` is write-1-to-clear. A student "clears" it with `EXTI->PR = 0;` and separately another writes `EXTI->PR |= (1U << 13);`. Explain what each statement actually does, and why the `|=` version is a subtle bug on a register with several pending lines.

6. Predict `EXTI->PR` bit 13 at each moment:

Moment	PR bit 13
before any press	
paused at a breakpoint inside `EXTI15_10_IRQHandler`	
paused at a breakpoint inside your callback	
back in `while (1)` after the press	

7. With the NVIC line disabled but everything else configured, a press occurs. State the value of `IMR` bit 13, `FTSR` bit 13, `PR` bit 13, and whether the handler runs.

Part C - Callback discipline and `volatile`

8. Each callback below violates at least one rule. Name every violation:

```c
/* (a) */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
```

```
    HAL_Delay(300);
}

/* (b) */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_13) {
        for (int i = 0; i < 3; i++) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
            HAL_Delay(100);
        }
    }
}
```

9. Explain, in terms of what the compiler is allowed to assume, why this loop can hang forever at `-O2` when `button_event` is not `volatile`:

```
while (button_event == 0) { }
```

10. In the consume pattern, why is the order

```
button_event = 0;
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
```

(clear **then** act) preferable to acting first? Consider a new press arriving during the toggle.

11. The lab declares `button_event` as `uint8_t` but `last_press_ms` as `uint32_t`. Justify both width choices.

Part D - Debounce

12. Falling edges arrive at t = 5000, 5003, 5008, 5012, 5240, 5310 ms. Trace the 80 ms guard: for each edge state accepted/rejected, the reason, and the value of `last_press_ms` afterwards. How many events does `main()` see?

13. Why is the guard's subtraction `(now - last_press_ms) >= 80` still correct when `HAL_GetTick()` wraps from `0xFFFFFFFF` to `0`? What property of unsigned arithmetic saves it?

14. Give one user-visible failure of choosing 5 ms for the guard, and one of choosing 500 ms — and say why 80 ms sits comfortably between them.

Part E - Evidence and behavior tests

15. Your claim: "the LED action runs in `main()`, not in interrupt context." Describe a two-breakpoint experiment that proves it.

16. Complete the expected-observation column, then add one test of your own that would expose a *missing debounce*:

Test	Expected observation
one slow press	
hold B1 for 3 seconds	
release B1	
ten quick presses	

17. A classmate's LED toggles on *release* instead of press. Name the single `.ioc` setting that explains it and the register bit that would confirm your diagnosis.

Exit self-assessment

Mark one:

☐ Ready for the L06 lab without additional review

☐ Need to review the path and its vocabulary
☐ Need to review the EXTI registers and write-1-to-clear
☐ Need to review `volatile` and the consume pattern
☐ Need to review the debounce arithmetic